

Unidad 1: Ingeniería de Software en Contexto.....	1
Contexto.....	1
La Crisis del Software.....	2
Disciplinas que conforman la Ingeniería de Software.....	2
Proyectos de software fallidos y exitosos.....	3
Ciclos de vida (Modelos de Proceso).....	4
Procesos. Empíricos vs. Definidos.....	4
Proyectos.....	6
PAPER: No Silver Bullet.....	9
PAPER: The Mythical Man-Month.....	11
Unidad 2: Gestión Lean-Ágil de Productos de Software.....	13
Contexto.....	13
Manifiesto Ágil.....	13
Requerimientos Ágiles.....	14
Product Backlog.....	15
Gestión de Producto.....	17
User Stories.....	18
Estimaciones Ágiles.....	20
Framework Scrum.....	21
Unidad 3: Gestión del Software como producto.....	24
Contexto.....	24
SCM.....	24

↑ Esos títulos y subtítulos los saqué del programa de la materia ↑

Unidad 1: Ingeniería de Software en Contexto

Contexto

¿Qué es el Software? → Conocimiento.

El software no es solo código, se tiende tener la missconception de que el software es solo el programa, pero es importante entender que el software es un set de programas y la documentación que acompaña. Saber programar nada más, no alcanza para hacer software.

El software, según Meles en clases, es CONOCIMIENTO. Es conocimiento / información que se nos presenta en distintos niveles de abstracción. Cada ARTEFACTO del PUD¹ es Software.

¿Qué es la Ingeniería de Software?

Disciplina que trabaja desarrollando saberes vinculados a las 4 dimensiones de la ingeniería.

Esta ingeniería es importante ya que:

- ★ Se necesitan soluciones rápidas, económicas y confiables.
- ★ A largo plazo, aplicar métodos de ingeniería de software reduce costos (la mayor parte surge de cambios durante la producción).

Además, la ingeniería de software se apoya en dos realidades, estas son:

- ★ Primero hay que **entender el problema** antes de desarrollar.
- ★ El **diseño** es fundamental: mejora la **calidad** y facilita el **mantenimiento**.

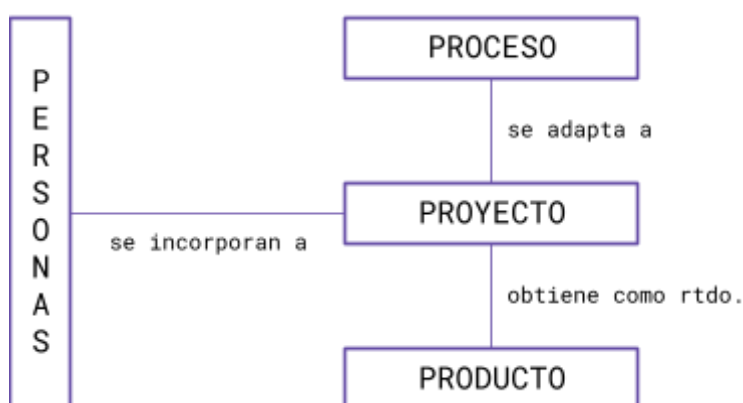
Las 4P del Desarrollo de Software

Estas cuatro P forman un marco integral:

Personas se refiere al equipo de desarrollo, usuarios y partes interesadas (es el recurso más valioso en el contexto del desarrollo de software);

Producto es el software a crear; **Proceso** es el conjunto de actividades

y flujos de trabajo para desarrollarlo; y **Proyecto** es la iniciativa que integra todos estos elementos para alcanzar un objetivo común.



¹Proceso Unificado de Desarrollo. Proceso Empírico que define los workflows de Modelado de Negocio, Requerimientos, Análisis, Diseño, Implementación, Pruebas, Despliegue.

La Crisis del Software.	Crisis del Software El término <i>Crisis del Software</i> fue planteado formalmente por Bauer en la Conferencia de la OTAN de 1968, aunque ya había sido mencionado antes por Dijkstra en “El humilde programador”. Se utilizó para describir las dificultades recurrentes en la producción de software: la incapacidad para generar productos libres de defectos, comprensibles y verificables. <u>Causas de la Crisis del Software</u> ★ Avance del hardware → sistemas más grandes y complejos. ○ Pero el software no evolucionó al mismo ritmo (es intangible, producto de actividad humana). ★ Demanda creciente de sistemas complejos. ★ Subestimación de la complejidad del desarrollo. ★ Cambios constantes en los productos para adaptarse al cliente. ★ Ausencia de disciplina formal en todos los aspectos de la producción de software. En la actualidad, aunque se dispone de nuevas herramientas y técnicas, muchos desarrollos de software siguen fracasando, mainly por <u>dos factores actuales</u>: ★ Demanda creciente → necesidad de nuevas técnicas de gestión y desarrollo. ★ Bajas expectativas → se conforman con software que “funcione”, sin calidad ni confiabilidad.
Disciplinas que conforman la Ingeniería de Software.	La Ingeniería de Software se estructura en distintas disciplinas que abarcan tanto las actividades técnicas vinculadas directamente con la construcción del software, como aquellas de gestión y soporte, que garantizan que el proceso y el producto tengan calidad y sean sostenibles en el tiempo. Disciplinas de gestión → Se refieren a la planificación, monitoreo y control del proyecto de desarrollo. Permiten coordinar tiempos, costos, alcance y calidad. ★ Planificación ★ Monitoreo y Control Disciplinas de soporte → Son transversales y actúan como garantes de la integridad y calidad del producto. Aquí lo central no es construir el software, sino mantenerlo controlado, medible y con calidad. ★ Gestión de configuración de software (SCM) → control de versiones y cambios. ★ Métricas de software → medición objetiva del proceso y del producto. ★ Aseguramiento de la calidad → garantizar que se cumplan estándares, normas y buenas prácticas.

Disciplinas técnicas → Son las que aportan directamente al desarrollo del software como producto.

Estas disciplinas están orientadas a “lo que se construye” y aseguran que el software cumpla con la función esperada. Incluyen:

- ★ Captura + análisis de requerimientos → comprensión del problema y de las necesidades del cliente.
- ★ Diseño de software → definición de la arquitectura y componentes.
- ★ Implementación → codificación propiamente dicha.
- ★ Pruebas → verificación y validación del sistema.
- ★ Despliegue → puesta en producción.
- ★ Documentación → soporte escrito para mantenimiento y uso.
- ★ Capacitación de usuarios → adopción y uso correcto del software.

Proyectos de software fallidos y exitosos.

El **éxito** de un proyecto de software no depende solo de programar bien, sino de la **forma en que se gestionan los requerimientos, la participación de los usuarios, la retroalimentación y el trabajo en equipo.**

Un software es exitoso cuando:

- ★ Los requerimientos son claros pero también flexibles, y maduran con el desarrollo.
- ★ Existe participación activa del usuario para esclarecer y validar lo que se necesita.
- ★ Se aplica retroalimentación constante, midiendo avances y corrigiendo fallas tempranamente.
- ★ El equipo está capacitado, motivado y comprometido.

Ejemplos de proyectos exitosos:

- ★ Linux, Amazon Web Services (AWS), Android.

En cambio, el software suele **fracasar** o volverse demasiado costoso cuando se **hacen las cosas al revés**: requerimientos incompletos o cambiantes, poca interacción con el usuario, escasos recursos, expectativas irreales o falta de apoyo gerencial.

Principales causas de fracaso (con % estimados):

- ★ Requerimientos incompletos → **13,1%**
- ★ Requerimientos cambiantes → **8,1%**
- ★ Poco involucramiento del usuario → **12,4%**
- ★ Recursos insuficientes → **10,6%**
- ★ Expectativas poco realistas/optimismo → **9,3%**
- ★ Falta de apoyo gerencial → **8,7%**

Ejemplos de proyectos fallidos:

- ★ Ubuntu Phone → intento fallido de crear un SO móvil basado en Linux.
- ★ FBI – Virtual Case File → fracaso para modernizar el sistema de casos.
- ★ Proyecto Taurus (Londres) → informatización fallida del mercado bursátil, con enormes sobrecostos.

Ciclos de vida (Modelos de Proceso)

El **ciclo de vida** es una **abstracción**: define cuáles son las **etapas** de un proyecto de software y en qué **orden** se ejecutan. No dice qué herramientas usar, quién hace qué, ni cómo ejecutar las actividades, sino sólo fases y secuencias.

Un proyecto tiene su propio ciclo de vida (comienza y termina con la entrega del producto), mientras que el producto generado tiene un ciclo más largo: continúa durante su mantenimiento hasta que finalmente se desecha.

1. **Secuencial** → Cada etapa se realiza **una vez y en orden**, sin retrocesos. Puede haber cierta devolución de información, pero es mínima. Ej.: Cascada.

Ventajas

- Simplicidad, fácil de entender y gestionar.
- Útil cuando los requerimientos están bien definidos desde el inicio.

Desventajas

- Poco flexible ante cambios.
- Riesgo alto si se detectan errores tarde (costoso corregir).

2. **Iterativo** → Basado en **repeticiones (iteraciones)** que generan **versiones mejoradas (incrementos)** del producto. Cada iteración capitaliza lo aprendido y ajusta el rumbo. Muy usado en procesos **empíricos**. Ej.: IeI

Ventajas

- Flexibilidad y adaptación a cambios.
- Retroalimentación continua.
- Se entrega valor al cliente de manera temprana.

Desventajas

- Mayor esfuerzo de coordinación.
- Riesgo de extenderse indefinidamente si no se controla bien.

3. **Rekursivo** → Pensado para **proyectos de alto riesgo**. Ej: Espiral, en B.

Ventajas

- Excelente manejo de riesgos.
- Permite focalizar en funcionalidades críticas.

Desventajas

- Complejo de gestionar.
- Puede demandar mucho tiempo y costo.

y su influencia en la Administración de Proyectos de Software. → El ciclo de vida es clave en la **gestión de proyectos** porque define el marco en el que se planifican, monitorean y controlan actividades. El **ciclo de vida del proyecto** termina con la entrega del producto.

y su relación con los Procesos de Desarrollo de Software → El **ciclo de vida** es la abstracción (define fases y orden). El **proceso de desarrollo** es la implementación concreta de ese ciclo, orientada a crear un producto.

Procesos. Empíricos vs. Definidos

¿Qué es un PROCESO?

Un proceso de software se define como un conjunto estructurado de actividades que, a raíz de un conjunto de entradas, producen una salida.

Las entradas no son solo los **requerimientos**, sino también las **personas** (principal recurso), junto con **materiales, hardware, librerías, herramientas, energía, etc.**

- **EEE:** proceso = secuencia de pasos ejecutados para un propósito. (Definición vaga).
- **CMM:** proceso = conjunto de actividades, métodos, prácticas y transformaciones que las personas usan para desarrollar o mantener software.

PROCESO DEFINIDO

Los procesos definidos surgen de una **concepción industrial**. Se inspiran en la lógica de las líneas de producción: si tengo los **mismos inputs**, aplico una serie de actividades fijas y siempre obtengo el **mismo output**. Bajo esta mirada, el proceso es predecible y repetible, lo que facilita el control y la gestión.

Esto funciona bien en industrias donde los productos son tangibles y las variables están estandarizadas, pero en el software resulta limitado. El desarrollo de sistemas rara vez es lineal y controlable en su totalidad, y la idea de que con los mismos insumos siempre obtendré el mismo resultado suele ser **difícil de sostener en la práctica**.

Un ejemplo clásico es el **Proceso Unificado de Desarrollo (PUD)**,

PROCESO EMPÍRICO

En contraste, los procesos empíricos se basan en el empirismo, es decir, en la **experiencia como fuente de conocimiento**. Reconocen que el desarrollo de software se da en **entornos complejos e inciertos**, donde no todas las variables pueden anticiparse o controlarse. Por eso, más que en la predictibilidad, confían en la **adaptación continua** y en la **capacidad de aprendizaje del equipo**.

La clave está en las personas: son ellas las que, con su experiencia, identifican problemas, ajustan el rumbo y capitalizan aprendizajes. La mejora no se produce ajustando un engranaje fijo del proceso (como en el modelo definido), sino mediante un ciclo constante de retroalimentación.

Este enfoque se organiza alrededor de **tres pilares**:

1. Transparencia → que todos tengan visibilidad de lo que ocurre.
2. Inspección → revisar periódicamente los avances y resultados.
3. Adaptación → modificar lo que haga falta en función de lo aprendido.

Un ejemplo típico es **Scrum**.

Proyectos

¿Qué es un PROYECTO? → Unidad de gestión de trabajo.

En otras definiciones, un proyecto puede entenderse como la **instanciación concreta de un proceso** dentro de un ciclo de vida. Mientras que el proceso es una **plantilla abstracta** que define qué actividades deben realizarse y en qué orden, el proyecto lo materializa adaptándolo a las necesidades específicas del contexto.

Un proyecto de software, al igual que cualquier proyecto, posee características esenciales:

- Tiene **duración limitada**: comienza y termina.
- Está **orientado a un objetivo claro y alcanzable**, lo cual lo hace único respecto de otros proyectos.
- Sus **resultados son únicos**: aunque se desarrollen sistemas similares, cada proyecto produce un resultado distinto.
- Implica un conjunto de **tareas interrelacionadas**, dependientes unas de otras, que requieren esfuerzos y recursos asignados. → **Elaboración Gradual**.

Administración de proyectos

La administración de proyectos se entiende como la **aplicación de conocimientos, habilidades, herramientas y técnicas** para ejecutar actividades que permitan cumplir con los requerimientos y entregar el producto esperado.

En el caso de proyectos dentro de procesos definidos, hablamos de **gestión tradicional de proyectos**: se basa en ejecutar de manera ordenada las actividades previamente establecidas, con el objetivo de garantizar la predictibilidad.

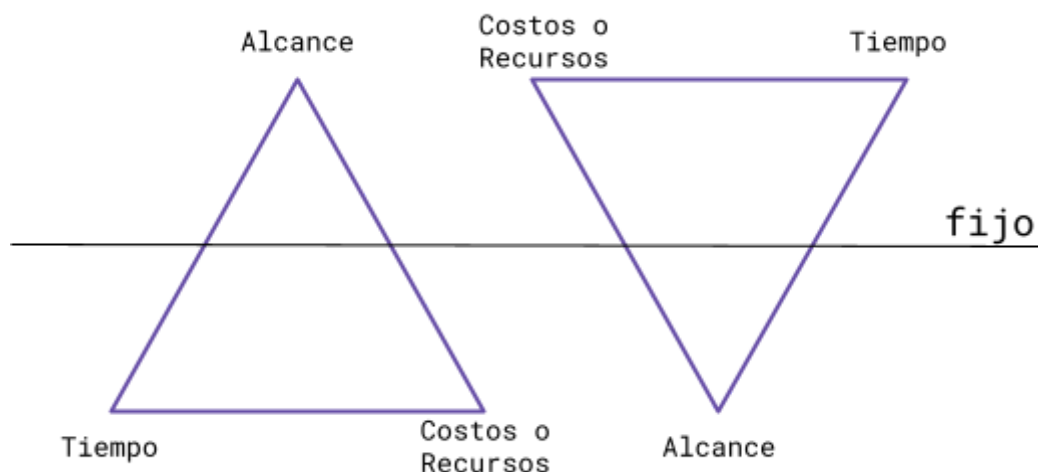
En esta gestión tradicional, los **roles** existentes son: un equipo compuesto por los miembros del proyecto y un líder de proyecto (Project Manager, PM), que es quien asume la responsabilidad de organizar, dirigir y controlar.

La Triple Restricción

Un concepto central en la gestión de proyectos. Plantea que todo proyecto debe gestionarse equilibrando tres variables críticas:

1. **Tiempo** (cronograma con fechas estipuladas).
2. **Presupuesto** (recursos económicos asignados).
3. **Alcance** (qué se va a construir, es decir, el objetivo del producto).

Estas variables están fuertemente interrelacionadas: si una se modifica, inevitablemente impacta en las otras. El rol del PM es **balancear estas restricciones** para que el proyecto logre sus objetivos, intentando evitar que un desajuste en el triángulo sacrifique la calidad.



La diferencia más marcada entre la gestión tradicional y la gestión Ágil se encuentra en la triple restricción: TRIÁNGULO DE HIERRO vs TRIÁNGULO ÁGIL

Enfoque Tradicional $\Delta \rightarrow$ Se fija el alcance como variable rígida. El tiempo y el costo se ajustan según lo que demande cumplir con ese alcance.

Ej.: si se definió que el proyecto debe entregar 3 pantallas, esas 3 pantallas deben estar completas, aunque haya que ampliar el tiempo o el presupuesto.

En un proyecto definido con ciclo de vida iterativo, lo que suele fijarse son los requerimientos (el alcance), y las iteraciones se arman en base a porciones de funcionalidad a desarrollar. Esto se conoce como Iteración de Alcance Fijo.

Agilismo $\nabla \rightarrow$ Se fija el tiempo como variable rígida. Lo importante es la entrega continua de valor, incluso si el producto no está finalizado en su totalidad.

Ej.: La iteración tiene una duración fija (ej.: 2 semanas), y ese día hay entrega sí o sí, aunque el alcance esté incompleto.

En un framework ágil (como Scrum), lo que se fija es el tiempo y el equipo de trabajo (lo que implica un costo estable), y lo que varía es el alcance: se entrega lo que se pueda priorizar dentro de la iteración, siempre asegurando calidad y valor. Esto se conoce como Iteración de Duración Fija.

Plan de Proyecto

El plan de proyecto funciona como una guía para el equipo y responde a:

- **¿Qué hacemos?** $\rightarrow$ Objetivo y Alcance.
 - **¿Cuándo lo hacemos?** $\rightarrow$ Calendarización.
 - **¿Cómo lo hacemos?** $\rightarrow$ Definir proceso, ciclo de vida y recursos.
 - **¿Quién lo hace?** $\rightarrow$ Personas. En el enfoque tradicional líder $\approx$ jefe.
- Incluiré para cada proyecto los roles que sean pertinentes para el mismo.

El plan incluye/define/documenta:

1. Alcance

Es todo el trabajo y solo el trabajo necesario para alcanzar el objetivo.

- *Alcance de producto*: características y funcionalidades (se mide en requerimientos → ERS).
- *Alcance de proyecto*: trabajo necesario para entregar el producto (se mide en tareas → Plan de Proyecto).

2. Proceso y ciclo de vida → definen las tareas y cómo se llevarán a cabo.

3. Estimaciones (enfoque tradicional) → tamaño, esfuerzo (horas ideales), costo y recursos críticos.

Estimar implica suponer sobre el futuro. Junto con la definición de requerimientos, es considerado uno de los procesos más complejos en el desarrollo de software

En el enfoque tradicional, la estimación suele recaer en el líder de proyecto, quien tiene la responsabilidad de calcular y planificar. En el agilismo, en cambio, la estimación es más colaborativa y se distribuye entre el equipo.

1. Tamaño (del producto)

- ★ En enfoques tradicionales: se mide en **Casos de Uso (CU) x Complejidad**. No alcanza con decir cuántos casos de uso hay: hay que ponderar la dificultad de cada uno.
 - Ejemplo: “100 CU complejos, 150 medianos, 50 simples”.
- ★ En enfoques ágiles: se mide con **Story Points**, una unidad relativa que representa el esfuerzo para completar una historia de usuario.

2. Esfuerzo en horas (del proyecto)

- ★ Se expresa en **horas/persona lineales**.
 - Ejemplo: 500 horas/persona significa que, en teoría, 1 persona tardaría 500 horas de trabajo continuo en completarlo.
- ★ “Lineales” porque se asume que las tareas no se ejecutan todas en paralelo, sino que requieren coordinación.

3. Tiempo/Calendario (del proyecto)

- ★ Se traduce el esfuerzo en una duración en días, semanas, meses o años.
- ★ Aquí se considera la **disponibilidad real del equipo**, la asignación de tareas y la capacidad de trabajar en paralelo.

4. Costos

- ★ Implica **monetizar el esfuerzo y los recursos**.
- ★ Incluye salarios, infraestructura, licencias, hardware, viajes, capacitación, etc.

5. Recursos Críticos

- ★ Aquellos recursos sin los cuales el proyecto no puede avanzar (ej.: un especialista clave, un servidor de pruebas, un software con licencia limitada).
- ★ Identificarlos temprano es fundamental para evitar cuellos de botella.

4. **Gestión de riesgos** → anticipar y mitigar problemas potenciales.

5. **Asignación de recursos** → distribuir personas y materiales.

6. **Calendarización** → programar tiempos y actividades.

7. **Definir métricas** → medición objetiva para evaluar progreso.

8. **Definir controles** → monitoreo y comparación de lo planificado con lo real.

PAPER: No Silver Bullet

El artículo "No Silver Bullet" fue escrito por Frederick P. Brooks Jr. en 1986. La idea principal es que no existe una única solución o un "remedio mágico" (la bala de plata) que por sí solo pueda resolver el problema fundamental del desarrollo de software y aumentar su productividad por diez en un periodo de diez años.

La complejidad inherente del software es el principal obstáculo.

La Distinción entre Esencia y Accidentes

Este es el concepto más importante del artículo. Brooks divide los problemas del desarrollo de software en dos categorías principales:

- **Esencia (Essential Complexity):** Son los problemas intrínsecos al software. Se refieren a la **complejidad fundamental** de lo que se está construyendo: las interacciones entre los componentes, la lógica, los algoritmos y la dificultad de conceptualizar un sistema. Esta complejidad no se puede eliminar, solo gestionar.
- **Accidentes (Accidental Complexity):** Son los problemas que no son inherentes al software, sino que **surgen del proceso de desarrollo y las herramientas utilizadas**. Incluyen la codificación, la depuración, la comunicación entre los equipos, el uso de lenguajes de programación obsoletos o la gestión de la documentación. Estos problemas sí se pueden reducir o eliminar con mejores herramientas y técnicas.

Brooks argumenta que durante años se ha hecho un gran progreso en la reducción de los problemas **accidentales** (por ejemplo, con lenguajes de alto nivel, entornos de desarrollo integrados, etc.), pero que estos avances no han impactado significativamente la **esencia** del problema, que sigue siendo la más difícil de abordar.

Puntos Clave sobre la Falta de una "Bala de Plata"

Brooks analiza varias tecnologías y enfoques que en su momento se consideraban prometedores, y explica por qué no son la "bala de plata" que se buscaba.

- **Lenguajes de Programación de Alto Nivel:** Han mejorado la productividad al reducir los errores y el tiempo de codificación (un problema **accidental**), pero no resuelven la complejidad intrínseca del diseño del software (la **esencia**).
- **Entornos de Programación y Herramientas (IDEs):** Similar a los lenguajes, estas herramientas facilitan el trabajo del programador (problema **accidental**) pero no cambian la dificultad de conceptualizar y diseñar un sistema.
- **Inteligencia Artificial (IA):** Brooks considera que la IA podría ayudar en la creación de prototipos o en la automatización de tareas, pero no en el diseño conceptual del sistema completo. La IA no puede adivinar la lógica de negocio ni las necesidades del cliente.
- **Programación Orientada a Objetos (POO):** Brooks reconoce el valor de la POO para la reusabilidad y la gestión de la complejidad, pero la considera una evolución valiosa, no la solución definitiva.
- **Programación de Prototipos:** Si bien permite una mejor interacción con el cliente para entender los requisitos, no reduce la complejidad inherente a la construcción del sistema final.

La Importancia de las Tácticas de Ataque

Dado que no hay una única solución mágica, se propone un conjunto de tácticas para enfrentar la complejidad esencial. Estas de verdad marcan la diferencia.

- **Comprar en lugar de construir:** Si hay un software comercial disponible que cumple con las necesidades, es más eficiente comprarlo que desarrollarlo desde cero.
- **Aprovechar los avances en las herramientas:** Usar herramientas que realmente simplifiquen las tareas repetitivas y **accidentales**.
- **Prototipado rápido:** Usar prototipos para iterar y refinar el diseño con el cliente antes de la implementación completa.
- **Desarrollo incremental:** Construir el software en partes pequeñas, liberando versiones funcionales gradualmente. Esto permite feedback temprano y gestión de la complejidad.
- **Grandes diseñadores (Programadores):** la habilidad y experiencia de los programadores y arquitectos de software es el factor más importante.

PAPER: The Mythical Man-Month

"The Mythical Man-Month" es una colección de ensayos escrita por Frederick P. Brooks Jr. en 1975. El título se traduce como **"El Hombre-Mes Mítico"** y el libro aborda los problemas de la gestión de proyectos de software. La idea principal es que la unidad de medida "hombre-mes" (la cantidad de trabajo que un hombre puede hacer en un mes) es una medida engañosa y, en la práctica, no funciona como se cree. El error común es pensar que el tiempo y el número de personas son directamente intercambiables.

Ley de Brooks

"Añadir personal a un proyecto de software retrasado lo retrasará aún más."

Esta frase resume el principal problema de la gestión de proyectos de software. ¿Por qué ocurre esto?

1. Costo de la comunicación: Cuando añades más personas a un equipo, el número de canales de comunicación aumenta exponencialmente. Cada nuevo miembro debe ser puesto al día, lo que consume tiempo valioso de los miembros actuales.
2. Divisibilidad del trabajo: No todas las tareas de un proyecto de software son fácilmente divisibles. Algunas partes, como el diseño y la arquitectura, requieren la colaboración de un grupo pequeño y coordinado. No puedes "parir un bebé en un mes con nueve mujeres". Es un proceso serial que no se puede acelerar simplemente agregando más recursos.
3. Tiempo de "rampa" (onboarding): Los nuevos miembros del equipo necesitan tiempo para aprender sobre el proyecto, su arquitectura, el código, los errores y la dinámica del equipo. Durante este tiempo, no son productivos y, de hecho, pueden disminuir la productividad general mientras hacen preguntas y consumen el tiempo de los miembros más antiguos.

Otros Conceptos Clave para la Gestión de Proyectos Mencionados en MMM

1. El hombre-mes como medida errónea

Brooks critica la idea de que se puede simplemente multiplicar el número de personas por el tiempo para estimar el esfuerzo total. Un proyecto de 100 "hombre-mes" no es lo mismo si lo hacen 100 personas en un mes que si lo hacen 10 en 10 meses. La complejidad de la interacción entre los programadores hace que las equivalencias sean falsas.

2. El rol del "arquitecto" (surgical team)

Brooks propone una analogía con la cirugía: un equipo de cirujanos tiene un cirujano principal y un equipo de apoyo que le asiste. De la misma manera, un equipo de desarrollo debería tener un **arquitecto o diseñador principal** que es el responsable de la integridad conceptual del sistema. Este líder talentoso y experimentado diseña y escribe el código crucial, mientras que el resto del equipo lo asiste con tareas menos críticas. Esto reduce la complejidad de la comunicación y asegura la coherencia del diseño.

3. La importancia de la integridad conceptual

Este es un concepto crucial. La integridad conceptual se refiere a la coherencia y armonía del diseño del sistema. Brooks argumenta que un buen sistema es aquel que es simple y conceptualmente elegante desde la perspectiva del usuario. Esto se logra cuando hay una visión unificada y un equipo de arquitectos (idealmente uno solo, o un grupo muy pequeño) que toma las decisiones de diseño. Un diseño con múltiples arquitectos tiende a ser incoherente y confuso.

4. La falta de comunicación y la documentación

Brooks subraya que la documentación es vital. La documentación del proyecto (especificaciones, manuales, etc.) es la principal herramienta para mantener la integridad conceptual y para que los nuevos miembros del equipo entiendan el sistema. Sin una buena documentación, los costos de comunicación se disparan.

Unidad 2: Gestión Lean-Ágil de Productos de Software

Contexto	Agilismo (ejercido por SCRUM) y Lean (ejercido por KANBAN) → NO son metodologías. SÍ son filosofías. (Meles lo repitió 8124821 veces en clases). El Agilismo surge del Manifiesto Ágil (2001) y representa una filosofía de gestión y desarrollo que prioriza: • La colaboración con el cliente por sobre la negociación contractual. • La adaptación al cambio en lugar de seguir estrictamente un plan. • El software funcionando más que la documentación exhaustiva. • Las personas y su interacción por sobre los procesos rígidos. Scrum es una forma concreta de ejercer esa filosofía ágil, organizada en iteraciones cortas (sprints), con roles definidos (PO, SM, Dev Team) y eventos que garantizan la transparencia, la inspección y la adaptación. El Lean Thinking nace en la industria automotriz (Toyota) y es también una filosofía de trabajo centrada en eliminar desperdicios, optimizar el flujo y generar valor continuo para el cliente. • Busca entregar lo justo en el momento justo, reduciendo tiempos muertos y exceso de tareas. • Promueve la mejora continua (kaizen). Kanban es una forma concreta de ejercer Lean en software, mediante tableros visuales que permiten gestionar el flujo de trabajo, limitar el trabajo en curso (WIP) y mejorar la eficiencia del equipo.
Manifiesto Ágil	El Agilismo (o filosofía ágil) nació hace unos 20 años, cuando un grupo de referentes de la industria se reunió en Utah y redactó el Manifiesto Ágil. No es una metodología ni un proceso, sino una ideología que busca un punto medio entre la ausencia total de procesos y el exceso de rigidez. Se apoya en los procesos empíricos, porque reconoce que en el desarrollo de software hay incertidumbre y cambios inevitables. La idea es trabajar en ciclos cortos de retroalimentación: comenzar con un producto mínimo viable (MVP), mostrarlo al cliente, recibir feedback y mejorar en cada iteración. Por eso, el agilismo solo tiene sentido en ciclos de vida iterativos, donde la adaptación y la flexibilidad son claves. El agilismo es una filosofía de gestión y desarrollo adaptativa. Los 4 valores del Manifiesto Ágil 1. Individuos e interacciones por sobre procesos y herramientas. 2. Software funcionando por sobre documentación extensiva. 3. Colaboración con el cliente por sobre negociación contractual. 4. Responder al cambio por sobre seguir un plan rígido.

Los **12 Principios Ágiles**

1. Prioridad: satisfacción del cliente con entregas tempranas y continuas.
2. Aceptar cambios de requerimientos, incluso tardíos.
3. Entregar software funcional con frecuencia.
4. Colaborar constantemente con el cliente.
5. Construir proyectos alrededor de personas motivadas.
6. Comunicación cara a cara como la más efectiva.
7. Medida principal de progreso: software funcionando.
8. Promover un ritmo de trabajo sostenible.
9. Búsqueda de excelencia técnica y buen diseño.
10. Simplicidad: no hacer más de lo necesario.
11. Equipos auto-organizados producen mejores resultados.
12. Reflexión y mejora continua en intervalos regulares.

Requerimientos Ágiles

En la gestión ágil, los requerimientos se entienden como **algo dinámico** que se va descubriendo de a poco y no como una lista cerrada desde el inicio del proyecto. La prioridad está en entregar **valor de negocio** de manera temprana y continua, y no en llenar el sistema de funcionalidades que después pueden quedar sin uso. De hecho, la experiencia demuestra que los usuarios suelen aprovechar solo alrededor del 20% de lo desarrollado, lo que hace evidente el problema de invertir demasiado esfuerzo en documentar y planificar de antemano.

Por esta razón, el enfoque ágil plantea que lo mejor es **definir sólo lo suficiente** para comenzar a trabajar, entregar un producto mínimo y recibir retroalimentación. El ciclo de feedback permite ajustar el rumbo, agregar o quitar funcionalidades, y asegurarse de que lo que se construye realmente tiene impacto.

El **Product Backlog** es la **lista priorizada y dinámica de todo lo que necesita un producto**: funcionalidades, mejoras o correcciones.

Lo gestiona el Product Owner, quien ordena los ítems por **valor de negocio**. Los que están arriba están más detallados y listos para el sprint; los de abajo pueden cambiar, crecer o descartarse.

Es la **fuentes única de requerimientos** en un proyecto ágil, siempre viva y en evolución.

El **Product Owner (PO)** es la figura clave en esta gestión. Es quien organiza los requerimientos en el **Product Backlog**, una lista priorizada y ordenada. Allí, los requerimientos más importantes se ubican en la parte superior y son los primeros en ser detallados y luego implementados. Los que quedan abajo de la pila pueden:

- ser removidos si ya no tienen sentido,
- subir si ganan importancia,
- o esperar hasta el momento en que alcancen la suficiente prioridad.

Esto se complementa con el concepto de **Just in Time (JIT)**: analizar en detalle los requerimientos únicamente cuando llega el momento de trabajarlos, ni antes (para evitar desperdicio) ni después (para no demorar la entrega).

La diferencia con el enfoque tradicional, es clara: allí se intenta especificar todo el producto al inicio, confiando en que el plan inicial se mantendrá estable. Ágil, en cambio, reconoce que:

- los cambios son inevitables,
- los usuarios muchas veces no saben lo que quieren hasta ver el sistema funcionando,
- y lo importante no es la completitud del plan, sino **dar valor de negocio con cada entrega**.

En ágil, los requerimientos suelen expresarse como **User Stories**: frases breves, en lenguaje natural, que describen quién necesita algo, qué necesita y qué valor le aporta. Estas historias se refinan a medida que suben en la pila y se acompañan con criterios de aceptación que delimitan cuándo la funcionalidad está cumplida.

En definitiva, los requerimientos ágiles buscan:

- **Maximizar valor**: priorizar lo que más impacto tiene.
- **Trabajar con el cliente** de manera continua.
- **Aceptar cambios** como parte natural del desarrollo.
- **Evitar desperdicio**, documentando y analizando sólo lo que se necesita en el momento adecuado.

Así, el software se convierte en una herramienta para **entregar valor**, y no en un fin en sí mismo.

Product Backlog

El **Product Backlog (PB)** es el **artefacto central de Scrum** que contiene todo lo que el producto debe tener para entregar valor al cliente. Se trata de una **lista priorizada, dinámica y viva** de características, mejoras, correcciones y requisitos no funcionales, organizada y gestionada por el **Product Owner (PO)**.

Cambia constantemente en función del aprendizaje, el feedback del cliente y las necesidades del negocio.

¿Qué meto en el Product Backlog? Se incluyen **los elementos suficientes para planear y ejecutar los próximos sprints**.

Los elementos del backlog se llaman **PBI (Product Backlog Items)**. PBIs posibles:

- **User Stories (US)**: ítems concretos listos para entrar en sprint (unidad mínima sugerida por Scrum).
- **Épicas**: historias demasiado grandes, que luego se dividirán.
- **Temas**: iniciativas amplias que agrupan épicas relacionadas.
- **Spikes**: investigaciones para reducir la incertidumbre.
- **Tech Stories (RNF)**: requisitos no funcionales o técnicos (seguridad, performance, escalabilidad). Estos no suelen estimarse de forma aislada, sino dentro de una US.

Dentro del Product Backlog **puede haber cualquier descripción de características del producto**. Lo que **no** se incluyen son **tareas**, porque las tareas corresponden al **Sprint Backlog**, donde los developers planifican el cómo.

Los PBIs tienen peso

Cada PBI debe ser **estimado**, ya sea en story points, tamaño relativo u otro método. Ese peso refleja su **complejidad, incertidumbre y esfuerzo**. De esta manera, el PO y el equipo pueden ordenar y planificar mejor.

Niveles de Abstracción en el Product Backlog

En un backlog no todos los ítems tienen el mismo nivel de detalle. El **grado de refinamiento** depende de cuán cerca está un ítem de ser implementado:

★ User Stories listas:

Ítems que cumplen los criterios INVEST, tienen criterios de aceptación claros y pueden ser implementados en un sprint.

Ejemplo: *Como estudiante quiero descargar mi certificado en PDF para poder presentarlo en mi trabajo.*

★ Épicas:

Historias de usuario muy grandes, que no pueden implementarse en un solo sprint. Se mantienen en el backlog como “contenedores” hasta que se refinan y dividen en historias más pequeñas.

Ejemplo: *Gestión completa de inscripciones online.*

★ Temas:

Nivel aún más alto que las épicas. Representan iniciativas amplias o propuestas de valor que agrupan varias épicas relacionadas. Funcionan como recordatorio de que en el futuro habrá que abordarlas.

Ejemplo: *Plataforma de servicios para estudiantes.*

★ Spikes (historias especiales):

Ítems creados **para investigar y reducir incertidumbre** antes de implementar una funcionalidad. No entregan valor directo al usuario, pero generan conocimiento útil. NO la estimamos. Simplemente la investigamos.

- **Spikes técnicas:** investigan tecnologías, arquitecturas o enfoques.

Ejemplo: probar compatibilidad de una API con un framework antes de integrarla.

- **Spikes funcionales:** exploran la interacción del usuario con el sistema.

Ejemplo: prototipo de flujo de inscripción y validación con usuarios reales.

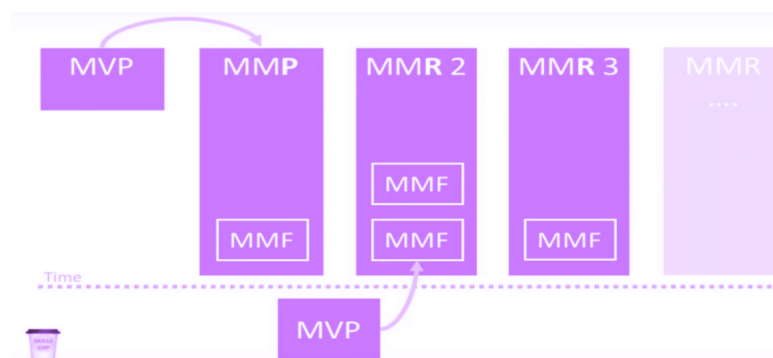
Gestión de Producto

En el desarrollo de software suele invertirse mucho esfuerzo en funcionalidades que rara vez se usan. Por eso, la gestión de producto busca **eliminar desperdicio** y enfocarse en entregar **valor real al usuario**.

La base está en construir lo necesario para que el software funcione, y el desafío es alcanzar la visión del producto sin perder tiempo en características innecesarias.

El proceso arranca con una **hipótesis de producto**: suposiciones sobre lo que el usuario necesita. Para validarla se utiliza el **MVP (Minimum Viable Product)**, que consiste en construir la versión más pequeña posible que permita probar si la hipótesis es correcta.

Una vez validada la idea, aparecen otros conceptos que ayudan a **planear la evolución del producto**:



★ **MMF (Minimum Marketable Feature)**

Conjunto mínimo de funcionalidades que hacen al producto atractivo para salir al mercado. Marca el punto en el que ya se puede comercializar.

★ **MVF (Minimum Viable Feature)**

Una funcionalidad mínima individual que puede implementarse rápidamente para comprobar su utilidad y valor agregado.

★ **MRF (Minimum Release Feature)**

El release más pequeño posible que ya entrega valor real a los usuarios. Es el incremento mínimo listo para entregar.

★ **MMP (Minimum Marketable Product)**

El producto mínimo completo que puede lanzarse al mercado. Va más allá del MVP, porque ya no busca validar, sino vender.

★ **MMR (Minimum Marketable Release)**

El release mínimo que contiene suficiente funcionalidad para ser comercializado. Se parece al MMP, pero enfocado en incrementos sucesivos de release más que en el producto como totalidad.

La clave de todos estos enfoques es el **feedback constante**: construir lo mínimo, ponerlo en manos de los usuarios, aprender de su reacción y ajustar. El éxito no es tener más características, sino entregar un producto que se use y aporte valor.

Otros conceptos/siglas:

★ **MLP (Minimum Lovable Product):**

No basta con que sea viable o comercializable: el producto debe ser **“amado” por los usuarios**. La idea es construir una versión mínima que genere **conexión emocional** y enganche al usuario, no solo que funcione.

MLP vs MVP

El MVP valida hipótesis con un prototipo funcional, mientras que el MLP busca que ese producto inicial ya sea lo suficientemente atractivo como para generar entusiasmo y fidelizar al usuario.

★ **MAP (Minimum Awesome Product):**

Similar al MLP, apunta a que el producto mínimo tenga un **factor diferencial atractivo** que lo haga destacar frente a la competencia.

★ **MSP (Minimum Sellable Product):**

Muy parecido al MMP, hace foco en lo **mínimo vendible**, es decir, lo suficiente para que alguien lo compre.

User Stories

Una **User Story** es una **técnica liviana para capturar requerimientos** desde la perspectiva de valor del usuario. No reemplaza especificaciones detalladas (como un Caso de Uso), sino que expresa intención, dispara conversación y permite planificar por valor en ciclos cortos.

La parte más importante no es lo escrito en la tarjeta, sino la conversación alrededor de la historia. Los errores en requerimientos (sobre todo no funcionales) se detectan tarde; por eso, la US busca reducir ese riesgo con feedback temprano.

Una US es una “porción vertical” → no es “solo UI” ni “solo base de datos”: se implementa end-to-end. Así entregás valor real en cada iteración.

Las 3 “C” de una US

- **Card (Tarjeta):** lo visible. Breve enunciado de rol-acción-valor.
- **Conversation (Conversación):** lo esencial y *no escrito*. Diálogo PO-equipo para aclarar detalles y límites.
- **Confirmation (Confirmación):** qué condiciones deben cumplirse para dar la US por hecha (se refleja en **criterios de aceptación** y **pruebas de aceptación**).

Formato (plantilla)

Como <rol> **quiero** <acción/actividad> **para** <valor de negocio>.

Nota: *El rol no es una persona fija*, es la **función** que ejecuta la acción.

Un “vendedor” puede tomar distintos roles según la tarea.

Buenas US: INVEST

- Independiente
- Negociable (no contrato rígido)
- Valiosa (valor claro para el negocio)
- Estimable
- Small (alcance acotado)
- Testable (criterios claros)

Antipatrones (no buenas US) frecuentes (y cómo evitarlos)

- **US demasiado grandes / vagos** → dividir (Small), concretar valor y criterios.
- **Diseño técnico embebido en la US** → los criterios deben ser **intención**, no solución.
- **Sin pruebas de aceptación** → difícil saber cuándo terminar.
- **No considerar RNF** → deuda invisible que explota más tarde.
- **No cortar vertical** → entregas que “no funcionan” de punta a punta.

Criterios de Aceptación vs. Pruebas de Aceptación

- **Criterios de Aceptación:** intención y límites desde la perspectiva del usuario. No describen la solución técnica; delimitan cuándo “está”.
- **Pruebas de Aceptación:** verificaciones concretas, muchas veces en estilo Given-When-Then (Gherkin). Pueden ir al dorso de la tarjeta o en el sistema.

Ejemplo (US “emitir factura A con CUIT validado”)

Criterios de aceptación (intención):

- *Debe validar CUIT contra AFIP antes de emitir.*
- *Si CUIT es inválido, impedir emisión y mostrar motivo.*
- *Registrar número de CAE y fecha en la factura.*

Pruebas de aceptación (verificación):

- **Dado** un CUIT válido **cuando** emito factura **entonces** se genera CAE y queda impresa la leyenda (PASA).
- **Dado** un CUIT inválido **cuando** intento emitir **entonces** la emisión se bloquea y veo el mensaje “CUIT inválido” (PASA).
- **Dado** un error de servicio AFIP **cuando** emito **entonces** el sistema reintenta 2 veces y registra incidente si falla (PASA).

Definition of Ready (DoR) vs. Definition of Done (DoD)

- **DoR** = “¿Está lista para empezar?”
Conjunto de criterios que una US debe cumplir **antes** de entrar al sprint. Evita arrancar con ambigüedad.
- **DoD** = “¿Está realmente terminada?”
Conjunto de criterios para considerar la US **finalizada** (lista para demo/release según tu política).

Definition of Ready (DoR)

Propósito: asegurar que el equipo puede implementar sin fricción. Se acuerda en equipo y suele expresarse como checklist.

Si algo falta → **refinement** o **spike** previo (timeboxed) para quitar incertidumbre.

Definition of Done (DoD)

Propósito: que “hecho” signifique lo mismo para todos, y que la calidad no se negocie.

Tu DoD puede tener niveles (equipo / producto / release). Definilos explícitamente.

Estimaciones Ágiles

¿Qué es estimar? Estimar significa **predecir el esfuerzo necesario** para desarrollar una funcionalidad en un contexto donde siempre existe **incertidumbre**. En ágil no buscamos exactitud matemática, sino una aproximación útil para planificar y priorizar.

Las personas somos mejores **comparando** que calculando en absoluto. Por eso, en vez de preguntar “¿cuántas horas lleva esto?”, se compara con otra referencia conocida.

En ambientes ágiles, la medida más común son los **Story Points (SP)**, que representan un **tamaño relativo** basado en:

- **Complejidad** de la historia.
- **Incetidumbre** o riesgo que conlleva.
- **Esfuerzo** (cantidad de trabajo o tiempo aproximado).

Se define primero una **user story canónica** (generalmente de tamaño pequeño y baja incertidumbre), que se toma como base con valor 1 o 2 SP. A partir de ahí, todas las demás historias se estiman en relación a ella.

Métodos de estimación basados en experiencia

1. **Datos históricos:** Comparar con proyectos o historias anteriores. Sirve incluso si el dominio es distinto, siempre que los datos estén bien estructurados.
2. **Juicio experto:**
 - **Puro:** un experto estima directamente.
 - **Delphi:** varios expertos estiman en rondas sucesivas y anónimas, hasta llegar a consenso.
3. **Analogía:** Comparar con casos pasados similares, seleccionando solo los más próximos al actual.

Técnicas ágiles específicas

Planning Poker (o Poker Estimation): Combina la opinión de los expertos con una desagregación en tareas más chicas.

Un principio que propone PP: “La persona idónea para una US, debería ser la persona que la implemente”

Se pone la US sobre la mesa y se empieza a debatir, a charlar. Luego el equipo elige una carta con un valor de la **serie de Fibonacci** (1, 2, 3, 5, 8...) para representar el peso de esa US.

- Si todos coinciden → se acepta esa estimación.
- Si difieren → se discuten las diferencias y se repite la votación hasta llegar a consenso.

La primera US que se estima, la llamaremos US CANÓNICA.

Características de la US canónica:

Debe tener una incertidumbre casi nula,
lo más pequeña posible (1 o 2),
y entendible por absolutamente todos los integrantes del equipo.

La US CANÓNICA debe entenderse por todos los miembros del equipo porque precisamente todos luego van a usarla como punto de comparación.

Framework Scrum

Scrum es un **marco de trabajo ligero** pensado para enfrentar proyectos complejos y adaptativos. No es una metodología cerrada, sino un marco que **combina empirismo y pensamiento Lean**. Esto significa que el equipo trabaja en ciclos cortos, inspecciona lo que va logrando y adapta lo necesario, siempre con foco en reducir desperdicio y entregar valor.

Otra definición: Es un proceso en el que se aplican de manera regular un conjunto de buenas prácticas para trabajar colaborativamente, en equipo y obtener el mejor resultado posible de proyectos.

Los **tres pilares** que sostienen Scrum son fundamentales porque garantizan aprendizaje continuo:

- **Transparencia** → la información relevante debe ser visible para todos los interesados.
- **Inspección** → el equipo revisa con frecuencia el progreso y el producto.
- **Adaptación** → cuando algo no va bien, se ajusta rápido el rumbo.

El éxito del framework no depende solo de reglas, sino de la cultura que adopte el equipo. Scrum propone **cinco valores**:

1. **Compromiso** → estar alineados con los objetivos y apoyarse mutuamente.
2. **Foco** → concentrarse en el trabajo del sprint, evitando distracciones.
3. **Franqueza** → hablar con honestidad sobre avances y problemas.
4. **Respeto** → reconocer las capacidades y aportes de cada miembro.
5. **Coraje** → enfrentar los desafíos y hacer lo correcto, incluso en contextos difíciles.

Equipo Scrum

El **equipo Scrum** es pequeño (máximo 10 personas) y no tiene jerarquías internas. La idea es que todos trabajen como **una unidad cohesionada** con responsabilidad compartida sobre el producto.

- **Developers:** crean los incrementos de producto utilizables en cada sprint.
- **Product Owner (PO):** responsable de maximizar el valor del producto; administra y prioriza el **Product Backlog**.
- **Scrum Master (SM):** vela por la correcta aplicación de Scrum y ayuda al equipo a ser más efectivo.

Eventos de Scrum

Cada sprint se organiza en torno a eventos que aseguran inspección, adaptación y transparencia.

★ Sprint:

Es el “contenedor” de todos los eventos. Tiene duración fija (máx. 1 mes) y siempre debe terminar con un incremento de valor.

Concepto de **TIME BOX** → es un período de tiempo fijo, inamovible y limitado, dentro del cual se debe realizar una actividad o producir un resultado. La idea central es que el tiempo no se negocia

- Un **Sprint** es un gran Time Box (1 mes o menos).
- La **Daily Scrum** es un micro Time Box (15 minutos).

★ Sprint Planning:

Marca el inicio del sprint. El equipo responde a tres preguntas clave:

- ¿Por qué este sprint es valioso?
- ¿Qué podemos hacer?
- ¿Cómo lo vamos a realizar?

★ Daily Scrum:

Reunión diaria de 15 minutos para que los Developers inspeccionen su progreso y ajusten el plan diario.

★ Sprint Review:

Se presenta el incremento a los interesados y se recogen sugerencias. El objetivo es inspeccionar resultados y definir adaptaciones.

★ Sprint Retrospective:

Último evento del sprint. El equipo reflexiona sobre el proceso, identifica mejoras y se compromete a aplicarlas en el siguiente sprint.

+ Refinamiento del Product Backlog:

Es una actividad on demand. Mantiene el PB organización, actualizado, etc

Unidad 3: Gestión del Software como producto

Contexto	Los sistemas evolucionan constantemente y cada cambio en el software genera nuevas versiones. Estas deben mantenerse y gestionarse para garantizar la integridad del producto. Aquí entra el SCM (Software Configuration Management).
SCM	El SCM es la disciplina que vela por la integridad del SW en todo momento. Disciplina protectora y transversal a todo el desarrollo del producto. El SCM aplica dirección y monitoreo administrativo y técnico para: ★ Identificar características técnicas y funcionales de los ítems de configuración. ★ Documentarlas para que queden registradas. ★ Controlar los cambios de esas características. ★ Registrar y reportar dichos cambios. ★ Verificar trazabilidad, asegurando correspondencia con los requerimientos. En otras palabras, permite que el producto conserve su integridad si: 1. Satisface las necesidades del usuario. 2. Permite rastrear de dónde vienen los cambios. 3. Cumple con criterios de performance y RNF. 4. Respeta las expectativas de costo. Términos Escenciales ★ Un ítem de configuración (IC) es cualquier artefacto del proyecto (código, documentación, script, manual). Puede cambiar y debe estar identificado. ★ El repositorio es el contenedor donde se guardan los IC y su historia, con atributos y relaciones. ★ Una versión es un estado puntual de un IC, mientras que una variante es una evolución paralela. ★ La configuración de software es el conjunto de todos los IC con su versión específica. ★ Una línea base (baseline) es un conjunto de IC revisados y aprobados, que sirven como referencia para futuros desarrollos. ★ Una rama (branch) es una bifurcación del desarrollo, usada para nuevas features, fixes o experimentos. En entornos ágiles, el SCM cambia su rol: no se trata de controlar

desarrolladores, sino de **dar soporte al equipo**.

- ★ Se trabaja con **documentación ligera (lean)** y trazabilidad simple.
- ★ El feedback debe ser **continuo y visible**, permitiendo ver la calidad e integridad del producto en cada paso.
- ★ La regla es **transparencia y baja fricción**: cuanto más automatizado (integración continua, pipelines, control de versiones), mejor.
- ★ La responsabilidad no es de un rol aislado, sino de **todo el equipo**.
- ★ Y finalmente, hay que **educar al equipo** en buenas prácticas de versionado, branching y control de cambios.